

Basic SQL

Importance of SQL in Relational Databases

- SQL is a major reason for the success of relational databases.
- Since SQL became a standard, users could migrate applications from older network or hierarchical databases to relational systems more easily.
- Although different DBMSs (e.g., MySQL, PostgreSQL, Oracle) have some variations, sticking to standard SQL ensures easier portability.

SQL vs. Relational Algebra

- Relational Algebra is a low-level procedural language (specifies how queries should be executed).
- SQL is a high-level declarative language (specifies what to retrieve, leaving optimization to the DBMS).
- SQL is influenced by tuple relational calculus but has a more user-friendly syntax.

History and Standardization of SQL

- Originally called SEQUEL (Structured English Query Language), developed by IBM for the SYSTEM R project.
- SQL became the standard language for relational DBMSs.
- Standardization is managed by ANSI (American National Standards Institute) and ISO (International Standards Organization).
- Major SQL standards:
 - SQL-86 (SQL1)
 - SQL-92 (SQL2) – major revision
 - SQL:1999 (SQL3) – introduced object-oriented features
 - SQL:2003, SQL:2006 – introduced XML support
 - SQL:2008 – added more object-oriented database features
 - SQL:2011 – latest update

SQL as a Comprehensive Database Language

SQL is a **full-fledged database language**, supporting:

- **Data Definition (DDL)**: Create and modify schemas and tables.
- **Data Manipulation (DML)**: Query, insert, update, delete records.
- **Security & Authorization**: Define user permissions.
- **Integrity Constraints**: Enforce primary keys, foreign keys, etc.
- **Transaction Control**: Manage concurrent database operations.
- **View Definition**: Create **virtual tables**.

SQL Data Definition and Data Types

- Schema and Catalog Concepts in SQL
- The CREATE TABLE Command in SQL

Schema and Catalog Concepts in SQL

SQL Schema:

- A **schema** is a logical structure that groups database objects such as **tables, views, types, constraints, and authorization permissions** under a common name.
- It helps in organizing related database elements.

Components of Schema:

- A schema includes:
- **Tables** – Stores actual data.
- **Views** – Virtual tables based on SQL queries.
- **Constraints** – Rules enforcing data integrity (e.g., primary keys, foreign keys).
- **Types & Domains** – Custom-defined data types.
- **Authorization Grants** – Controls access to schema elements.

Creating a Schema

- The **CREATE SCHEMA** statement allows you to define a schema and assign an owner (authorization identifier).

Example:

```
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
```

- This statement:
Creates a **schema** named COMPANY.
Assigns ownership to **user** Jsmith.

Defining Schema Elements

- Schema elements can be defined at the time of schema creation or later.

Example:

```
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith'  
    CREATE TABLE EMPLOYEE (  
        EmpID INT PRIMARY KEY,  
        Name VARCHAR(50),  
        Salary DECIMAL(10,2)  
    );
```

This creates the EMPLOYEE table inside the COMPANY schema

SQL Catalog

- A catalog is a collection of schemas in a database installation.
- It provides a higher level of organization for managing multiple schemas.

Features of Catalogs

- Each **catalog** contains **multiple schemas**.
- A database typically has **one default catalog**.
- Every catalog has a built-in schema **INFORMATION_SCHEMA**, which stores metadata about schemas.

Database
Table 1
Table 2
Table 3

Database
Table 1
Table 2
Table 3

Database > Table

Catalog				
<table> <tr> <th>Schema</th></tr> <tr> <td>Table 1</td></tr> <tr> <td>Table 2</td></tr> <tr> <td>Table 3</td></tr> </table>	Schema	Table 1	Table 2	Table 3
Schema				
Table 1				
Table 2				
Table 3				
<table> <tr> <th>Schema</th></tr> <tr> <td>Table 1</td></tr> <tr> <td>Table 2</td></tr> <tr> <td>Table 3</td></tr> </table>	Schema	Table 1	Table 2	Table 3
Schema				
Table 1				
Table 2				
Table 3				

Catalog > Schema > Table

Viewing Schema Information

To check existing schemas in a catalog:

```
SELECT schema_name FROM INFORMATION_SCHEMA.SCHEMATA;
```

To see tables in a schema:

```
SELECT table_name FROM INFORMATION_SCHEMA.TABLES WHERE  
table_schema = 'COMPANY';
```

Referential Integrity and Schema Constraints

- **Foreign key constraints** require that related tables exist in the **same catalog**
- Schemas in the same catalog can **share types and domain definitions**.

Advantages of Using Schemas and Catalogs

- Better Organization: Groups related tables, constraints, and views.
- Security Management: Assigns permissions at the schema level.
- Easier Data Migration: Moves schemas between databases with minimal changes.
- Supports Multi-Tenant Databases: Different applications can have separate schemas within the same database.

Attribute Data Types and Domains in SQL

The basic data types available for attributes include numeric, character string, bit string, Boolean, date, and time.

1. Numeric data types: Include integer and floating-point numbers.

a. Integer Types:

- **INTEGER (INT)** – Standard integer type.
- **SMALLINT** – Smaller-sized integer type.

b. Floating-Point Types:

- **FLOAT (REAL)** – Floating-point number with varying precision.
- **DOUBLE PRECISION** – Higher precision floating-point number.

Integer Types

c. Formatted Numbers:

- Declared using **DECIMAL(i, j)**, **DEC(i, j)**, or **NUMERIC(i, j)**.
- **i (Precision)** – Total number of decimal digits.
- **j (Scale)** – Number of digits after the decimal point.

Example: **DECIMAL(5,2)** or **NUMERIC(5,2)**

Can store numbers with 5 total digits, 2 digits after the decimal point.

Example values: 123.45, 9.99, 100.00

Invalid value: 1234.567 (exceeds 5 digits in total).

d. Defaults:

Default Scale: Zero (i.e., no decimal places).

Default Precision: Implementation-defined (varies by database system).

Character-string data types

Types of Character Strings:

- Fixed-Length Strings: CHAR(n) or CHARACTER(n), where n is the number of characters.
- Variable-Length Strings: VARCHAR(n), CHAR VARYING(n), or CHARACTER VARYING(n), where n is the maximum length.

String Literals

- Enclosed in single quotation marks ('example').
- Case-sensitive (e.g., 'abc' and 'ABC' are different).

Large Text Storage (CLOB)

CHARACTER LARGE OBJECT (CLOB) is used for large text values (e.g., documents).

Maximum length can be specified in kilobytes (K), megabytes (M), or gigabytes (G).

Example: CLOB(20M) allows up to 20 MB of text

Data types

Boolean Data Type:

Can store **TRUE**, **FALSE**, or **UNKNOWN** (due to NULL values in SQL).

DATE Data Type:

- Stores year, month, and day in the format YYYY-MM-DD.
- Example: DATE '2008-09-27'.
- Supports comparisons (<, >, =)—earlier dates are considered smaller.

TIME Data Type

Stores hour, minute, and second in the format HH:MM:SS.

Example: TIME '09:12:47'.

TIME(i) allows fractional seconds (i positions after .).

Example: TIME(3) '09:12:47.123'.

TIME WITH TIME ZONE:

Includes time zone offset (from +13:00 to -12:59).

Example: TIME '09:12:47+05:30'.

If WITH TIME ZONE is not specified, it defaults to the SQL session's local time zone.

TIMESTAMP Data Type:

Stores both DATE and TIME, including fractional seconds.

Example: TIMESTAMP '2008-09-27 09:12:47.648302'.

WITH TIME ZONE can be included for global time handling.

INTERVAL Data Type

Stores a relative time period to increment or decrement date, time, or timestamp.

Two types:

YEAR/MONTH interval (e.g., INTERVAL '2-3' YEAR TO MONTH → 2 years, 3 months).

DAY/TIME interval (e.g., INTERVAL '5 12:30:15' DAY TO SECOND → 5 days, 12 hours, 30 minutes, 15 seconds).

Specifying Constraints in SQL

Attribute Constraints and Defaults in SQL:

NOT NULL Constraint:

- Prevents an attribute from having **NULL** values.
- **Primary key attributes are always NOT NULL.**

DEFAULT Clause:

Specifies a default value if no explicit value is provided.

If no default is set, the default value is NULL (unless NOT NULL is enforced).

CHECK Constraint:

- Restricts attribute values based on a condition.
- Example (restricting department numbers to 1-20)

CREATE DOMAIN with CHECK:

Defines a custom data type with restrictions. Example (creating a domain for department numbers),

```
CREATE DOMAIN D_NUM AS INTEGER  
CHECK (D_NUM > 0 AND D_NUM < 21);
```

Specifying Key and Referential Integrity Constraints

PRIMARY KEY Constraint:

- Defines one or more attributes as the primary key of a table.
- Ensures that the attribute(s) must be unique and NOT NULL.

```
CREATE TABLE Department (  
    Dnumber INT PRIMARY KEY  
);  
  
CREATE TABLE Works_On (  
    EmpID INT,  
    ProjectID INT,  
    PRIMARY KEY (EmpID, ProjectID)  
);
```

UNIQUE Constraint (Alternate Key)

- Ensures that an attribute (or combination) has **unique values**, but allows **NULLs**.

```
CREATE TABLE Department (  
    Dnumber INT PRIMARY KEY,  
    Dname VARCHAR(15) UNIQUE -- Unique department name  
);
```

FOREIGN KEY (Referential Integrity):

- Ensures that a foreign key value refers to an existing primary key in another table.
- Prevents inserting an invalid reference and deleting referenced records.

```
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    Dno INT,  
    FOREIGN KEY (Dno) REFERENCES Department(Dnumber)  
);
```

Handling Referential Integrity Violations:

- By default, SQL rejects updates that violate integrity (RESTRICT).

Alternative actions:

SET NULL → Foreign key is set to NULL if referenced record is deleted.

CASCADE → Changes are cascaded to dependent records (deletion or update).

SET DEFAULT → Foreign key is set to a default value.

```
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    Super_ssn INT,  
    FOREIGN KEY (Super_ssn) REFERENCES Employee(EmpID)  
    ON DELETE SET NULL  
    ON UPDATE CASCADE  
);
```

When to Use CASCADE?

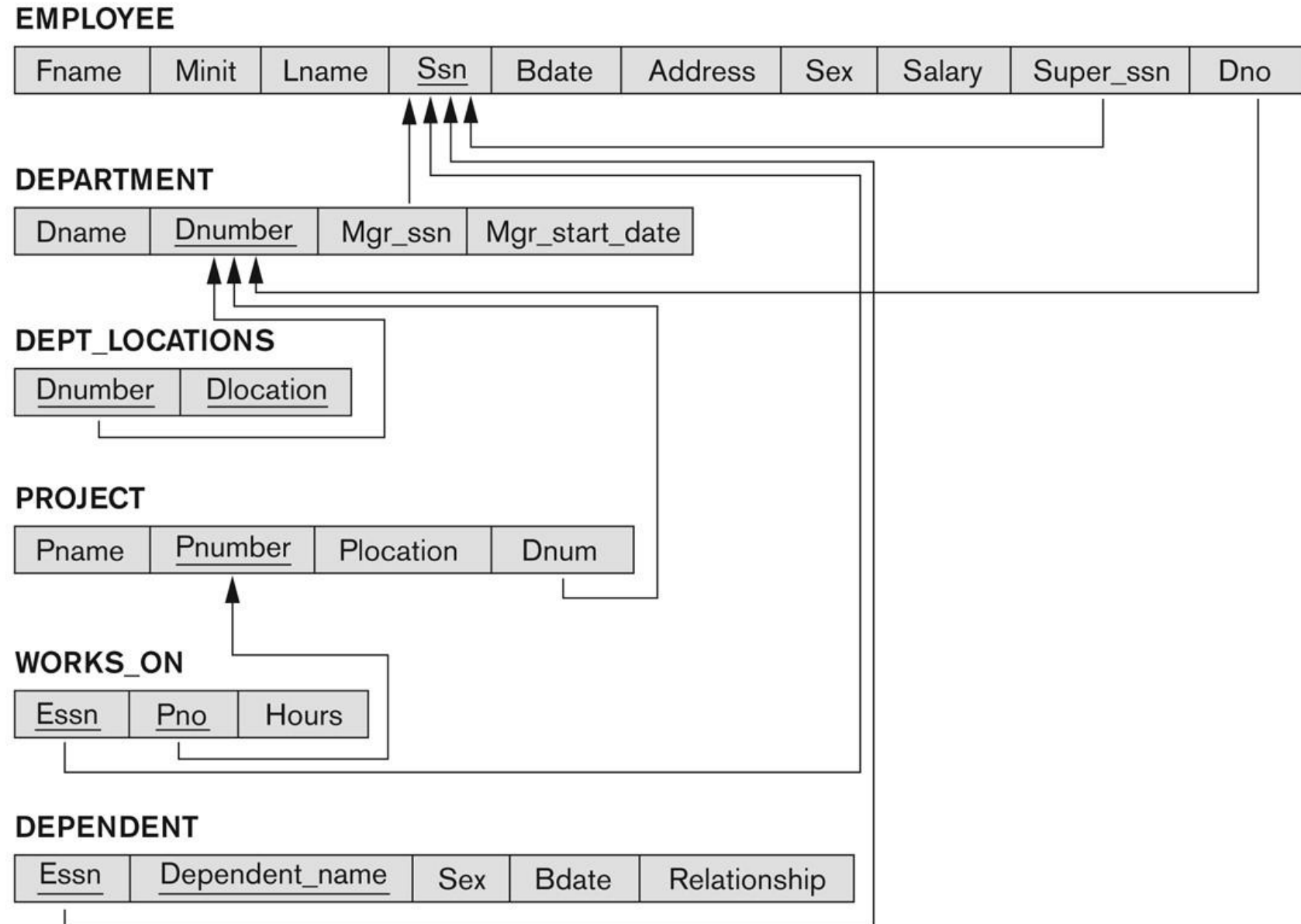
- Suitable for relationship tables (e.g., Works_On).
- Used for multivalued attributes (e.g., Dept_Locations).
- Required for weak entities (e.g., Dependent).

The CREATE TABLE Command in SQL

- The CREATE TABLE command is used to specify a new relation by giving it a name and specifying its attributes and initial constraints.
- The attributes are specified first, and each attribute is given a name, a data type to specify its domain of values, and any attribute constraints, such as NOT NULL.
- The key, entity integrity, and referential integrity constraints can be specified within the CREATE TABLE statement after the attributes are declared.
- The relations declared through CREATE TABLE statements are called **base tables**.
- Base table means The relation and its tuples are actually created and stored as a file by the DBMS

Figure 5.7

Referential integrity constraints displayed on the COMPANY relational database schema.



Examples for creating relation using create table command.

CREATE TABLE EMPLOYEE

(Fname	VARCHAR(15)	NOT NULL,
Minit	CHAR,	
Lname	VARCHAR(15)	NOT NULL,
Ssn	CHAR(9)	NOT NULL,
Bdate	DATE,	
Address	VARCHAR(30),	
Sex	CHAR,	
Salary	DECIMAL(10,2),	
Super_ssn	CHAR(9),	
Dno	INT	NOT NULL,

PRIMARY KEY (Ssn),

FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn),

FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber));

CREATE TABLE DEPARTMENT

(Dname	VARCHAR(15)	NOT NULL,
Dnumber	INT	NOT NULL,
Mgr_ssn	CHAR(9)	NOT NULL,
Mgr_start_date	DATE,	

PRIMARY KEY (Dnumber),

UNIQUE (Dname),

FOREIGN KEY (Mgr_ssn) **REFERENCES** EMPLOYEE(Ssn));

CREATE TABLE DEPT_LOCATIONS

(Dnumber	INT	NOT NULL,
Dlocation	VARCHAR(15)	NOT NULL,

PRIMARY KEY (Dnumber, Dlocation),

FOREIGN KEY (Dnumber) **REFERENCES** DEPARTMENT(Dnumber));

CREATE TABLE PROJECT

(Pname	VARCHAR(15)	NOT NULL,
Pnumber	INT	NOT NULL,
Plocation	VARCHAR(15),	
Dnum	INT	NOT NULL,

PRIMARY KEY (Pnumber),

UNIQUE (Pname),

FOREIGN KEY (Dnum) **REFERENCES** DEPARTMENT(Dnumber));

CREATE TABLE WORKS_ON

(Essn	CHAR(9)	NOT NULL,
Pno	INT	NOT NULL,
Hours	DECIMAL(3,1)	NOT NULL,

PRIMARY KEY (Essn, Pno),

FOREIGN KEY (Essn) **REFERENCES** EMPLOYEE(Ssn),

FOREIGN KEY (Pno) **REFERENCES** PROJECT(Pnumber));

CREATE TABLE DEPENDENT

(Essn	CHAR(9)	NOT NULL,
Dependent_name	VARCHAR(15)	NOT NULL,
Sex	CHAR,	
Bdate	DATE,	
Relationship	VARCHAR(8),	

PRIMARY KEY (Essn, Dependent_name),

FOREIGN KEY (Essn) **REFERENCES** EMPLOYEE(Ssn));

INSERT, DELETE, and UPDATE Statements in SQL In SQL

Three commands can be used to modify the database:

INSERT, DELETE, and UPDATE.

The INSERT Command:

- It used to add a single tuple to a relation.
- We must specify the relation name and a list of values for the tuple. The values should be listed in the same order in which the corresponding attributes were specified in the CREATE TABLE command.

```
INSERT INTO    EMPLOYEE
VALUES        ( 'Richard', 'K', 'Marini', '653298653', '1962-12-30', '98
                  Oak Forest, Katy, TX', 'M', 37000, '653298653', 4 );
```


example

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

```
INSERT INTO      EMPLOYEE  
VALUES          ( 'Richard', 'K', 'Marini', '653298653', '1962-12-30', '98  
                  Oak Forest, Katy, TX', 'M', 37000, '653298653', 4 );
```

```
INSERT INTO      EMPLOYEE (Fname, Lname, Dno, Ssn)  
VALUES          ('Richard', 'Marini', 4, '653298653');
```

The DELETE Command

- The DELETE command removes tuples from a relation.
- It includes a WHERE clause.
- the deletion may propagate to tuples in other relations if referential triggered actions are specified in the referential integrity constraints of the DDL.
- Depending on the number of tuples selected by the condition in the WHERE clause, zero, one, or several tuples can be deleted by a single DELETE command.
- A missing WHERE clause specifies that all tuples in the relation are to be deleted.
- DROP TABLE command to remove the table definition.

Example

**DELETE FROM
WHERE**

EMPLOYEE
Lname='Brown';

**DELETE FROM
WHERE**

EMPLOYEE
Ssn='123456789';

**DELETE FROM
WHERE**

EMPLOYEE
Dno=5;

DELETE FROM

EMPLOYEE;

The UPDATE Command

- The UPDATE command is used to modify attribute values of one or more selected tuples.
- a WHERE clause in the UPDATE command selects the tuples to be modified from a single relation.
- updating a primary key value may propagate to the foreign key values of tuples in other relations if such a referential triggered action is specified in the referential integrity constraints of the DDL.
- An additional SET clause in the UPDATE command specifies the attributes to be modified and their new values

Example

to change the location and controlling department number of project number 10 to 'Bellaire' and 5

```
UPDATE    PROJECT
SET       Plocation = 'Bellaire', Dnum = 5
WHERE     Pnumber=10;
```

Increase the salary of the employee by 5000/- belongs to reseach.

```
UPDATE EMPLOYEE SET Salary=salary+5000 where Dno=5;
```

Alter Command

- The **ALTER command** is used to **modify the structure of an existing table**.
- ALTER is used to change an already created table

Using ALTER, you can:

- Add a new column
- Modify a column
- Delete a column
- Rename a column
- Add or remove constraints

Syntax:

```
ALTER TABLE table_name action;
```

Alter command

Add a New Column:

```
ALTER TABLE student ADD age INT;
```

Modify a Column:

```
ALTER TABLE student MODIFY age INT NOT NULL;
```

Rename a Column:

```
ALTER TABLE student RENAME COLUMN name TO student_name;
```

Drop (Delete) a Column:

```
ALTER TABLE student DROP COLUMN age;
```

WHERE Clause

The **WHERE clause** is used to **filter records** in a query.

WHERE is used to select only those rows that satisfy a given condition.

Syntax:

```
SELECT column_name FROM table_name WHERE condition;
```

Why We Use WHERE?

- To get **specific data**
- To avoid displaying all records
- To apply conditions

Examples

Select Specific Row:

```
SELECT * FROM student WHERE id = 101;
```

Condition with Numbers:

```
SELECT name FROM student WHERE marks > 80;
```

Condition with Text:

```
SELECT * FROM student WHERE dept = 'CSE';
```

Using Operators

Comparison Operators:

- = → equal
- != or <> → not equal
- → greater than
- < → less than
- >=, <=

Logical Operators:

Multiple conditions:

- AND
- OR
- NOT

Special Conditions:

- BETWEEN
- IN
- LIKE (Pattern Matching)
- IS NULL